# Experiment 8

**Student Name: Chandan Kumar** **UID: 23BAI70147**
**Branch: BE CSE** **Section/Group: 23AIT_KRG-G- 1**
**Semester: 6th** **Date of Performance: 10/04/26**
**Subject Name: Full Stack - II** **Subject Code: 23CSH - 382**

1. **Aim:**
   App Security & Full-Stack Integration (LivePoll System)

2. **Objective:**

   By the end of this experiment, students will be able to:

   - To get familiar with the basics of Spring Security and why it is important in backend applications
   - To understand how requests are handled internally using the security filter chain
   - To learn the difference between:
     o Authentication (verifying user identity) o Authorization (controlling access to resources)

3. **Implementation/Code:**

   **BackendApplication.java:**

```java
package com.example.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BackendApplication {

    public static void main(String[] args) {
        SpringApplication.run(BackendApplication.class, args);
    }  Tab to Jump

}
 Ctrl+L for Agent
```

**Poll.java:**

```java
package com.example.livepoll.entity;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "polls")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Poll {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String question;

    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "poll_options", joinColumns = @JoinColumn(name = "poll_id"))
    @Column(name = "option_text")
    private List<String> options = new ArrayList<>();

    @Column(nullable = false)
    private Boolean active;

    @Column(nullable = false)
    private String createdBy;

    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        if (this.active == null) this.active = true;
    }
}
```

**Role.java:**

```java
package com.experiment9.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "roles")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Enumerated(EnumType.STRING)
    @Column(length = 20, unique = true, nullable = false)
    private ERole name;
}
```

## User.java:

```java
package com.example.livepoll.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.*;

import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "users",
        uniqueConstraints = {
            @UniqueConstraint(columnNames = "username"),
            @UniqueConstraint(columnNames = "email")
        })
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Size(max = 50)
    private String username;

    @NotBlank
    @Size(max = 100)
    @Email
    private String email;

    @Size(max = 120)
    private String password;

    private String provider;   // "local" or "google"
    private String providerId; // Google sub-id

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles",
               joinColumns = @JoinColumn(name = "user_id"),
               inverseJoinColumns = @JoinColumn(name = "role_id"))
    private Set<Role> roles = new HashSet<>();
}
```

## 4. Learning Outcomes:

1. **Explain the fundamentals of application security** and its importance in real-world full-stack systems.
2. **Differentiate between authentication and authorization**, and understand how both are implemented in backend applications.
3. **Configure and customize Spring Security** to protect REST APIs and manage secure request handling.
4. **Understand and implement the Spring Security filter chain**, and analyze how requests are processed and validated.
5. **Integrate OAuth 2.0 (Google Login)** for secure third-party authentication in modern web applications.
6. **Implement Role-Based Access Control (RBAC)** to restrict access to resources based on user roles such as USER and ADMIN.
7. **Secure endpoints using annotations** like `@PreAuthorize` and apply fine-grained access control.
8. **Establish secure full-stack communication** by integrating a React frontend with a Spring Boot backend.
9. **Configure and manage CORS policies** to enable safe cross-origin communication.
10. **Implement token-based authentication flow** (JWT/OAuth), including token storage, validation, and secure request handling.
11. **Analyze the complete authentication flow**, from user login to backend validation and role-based access decisions.
12. **Develop and evaluate secure full-stack applications**, identifying potential security risks and applying best practices.